



Smart Contract Audit Report

2026-05-16

Produced for

Alpha Brands Group HK Limited

B, 3/F, One Jardine's Lookout
1 Wang Tak St,
Happy Valley

1. Scope

This report presents a security review of the smart contract code in the `videa-pay` repository, focused on the Solidity contracts under `contracts/contracts/`.

The current review baseline is:

Client	Alpha Brands Group HK Limited
Client address	B, 3/F, One Jardine's Lookout 1 Wang Tak St, Happy Valley
Repository	videa-pay
Initial audit commit	2ba60dbee490111a87b4b7e09b30a455d09da5ce
Remediation review commit	ac21c55f58a66df81f4c9b6dfa463f690cebc1c4
Fix range reviewed	2ba60dbe..ac21c55f
Report status	Final
Primary contract	contracts/contracts/VimoVault.sol
Solidity compiler	0.8.24, optimizer enabled, 200 runs, viaIR=true
Primary dependencies	OpenZeppelin Contracts / Contracts Upgradeable 5.4.0 resolved in lockfile; Permit2 interface

The initial findings in this report were produced against 2ba60dbe. Remediation statuses are based on a targeted review of the project changes through ac21c55f. The remediation review focused on the reported findings, their directly related code paths, and the new tests or tooling output relevant to those fixes; it should not be read as a full re-audit of every application component changed in the repository.

The main in-scope contract is `VimoVault`, an upgradeable vault used for stablecoin deposits, relayer-executed purchases, creator/distributor/platform payment distribution, and emergency withdrawals.

1.1 In-Scope Files

- `contracts/contracts/VimoVault.sol`
- `contracts/contracts/permit2/IAAllowanceTransfer.sol`

2. Out of Scope

- Frontend, backend, relayer service, database, or off-chain accounting code outside the smart contract project.
- Private keys, operational procedures, multisig signer security, or production incident response processes.
- Economic, legal, tax, sanctions, compliance, or business-model analysis.
- Third-party token implementations beyond their expected ERC-20 interaction surface.
- Deployment, upgrade, verification, and admin scripts under `contracts/scripts/`.
- Tests, fuzzing harnesses, and security-tool harnesses.
- Hardhat, Slither, Echidna, Medusa, and other project configuration files.
- Live deployment state, block explorer verification, or historical on-chain events unless deployment addresses are later provided.

3. Executive Summary

Severity	Critical	High	Medium	Low	Informational
Count	0	0	0	5	2

4. System Overview

4.1 Architecture

VimoVault is a UUPS-upgradeable contract using OpenZeppelin upgradeable modules for access control, pausing, reentrancy protection, and EIP-712 typed data verification.

Complete functional map

1. Deployment initializes the proxy with an admin, relayer, platform wallet, and platform-fee basis points. The initializer grants default admin, admin, relayer, and pauser roles.
2. Admins configure the payment surface by adding or removing supported tokens, updating the platform wallet, updating the platform fee, replacing the relayer set, enabling or disabling emergency withdrawals, and authorizing UUPS upgrades.
3. Pausers can pause and unpause the main business flows. Deposits and relayer purchase execution are pause-gated; emergency withdrawals are intentionally not pause-gated.
4. Users deposit supported ERC-20 tokens through Permit2. First-time or refreshed Permit2 authorization can be registered through `permitAndDeposit`; users with an existing Permit2 allowance can call `deposit`. The vault credits per-user virtual balances and aggregate token deposits based on actual tokens received.
5. A privileged relayer executes purchases through `executePurchase`. The vault verifies the user's EIP-712 authorization, checks replay protection, debits the user's virtual balance, optionally registers a fan-creator-distributor relationship, and transfers funds to creator, distributor, and platform recipients.
6. Users can execute an immediate emergency withdrawal for their full recorded balance of a token when withdrawals are enabled. This path bypasses pause so users can exit during an emergency.
7. Role administration is also exposed through inherited AccessControl functions. Default-admin renunciation is blocked, but role grants and revocations remain available according to the OpenZeppelin role-admin model.
8. Read-only surfaces expose balances, token support, aggregate deposits, executed-action status, relationship state, role membership, platform parameters, pause state, and upgrade-related interface support.

4.2 External Entry Points

This map treats contract-defined functions, inherited public functions, and compiler-generated public getters as part of the externally reachable contract surface.

- **Initialization:** `initialize` sets roles, platform wallet, platform fee, and inherited module state. The implementation constructor disables direct initialization.
- **Deposits:** `permitAndDeposit` and `deposit` pull tokens through the canonical Permit2 address and credit `userDeposits` plus `totalDeposits`.
- **Payments:** `executePurchase` is restricted to `RELAYER_ROLE`; it validates the user's signature, marks the action ID executed, updates accounting, optionally records relationship data, and distributes token balances.
- **Relationships:** There is no standalone registration function. Relationships are registered inline during `executePurchase` when signed distributor parameters are valid and no active relationship already exists.
- **Emergency withdrawals:** `executeEmergencyWithdrawal` transfers the caller's full recorded token balance back to the caller when `withdrawalsEnabled` is true.
- **Contract administration:** The external setters for withdrawal enablement, supported tokens, platform wallet, platform fee, and relayer replacement are restricted to `ADMIN_ROLE`.
- **Circuit breaker:** `pause` and `unpause` are restricted to `PAUSER_ROLE`.

- **Role administration:** Inherited `grantRole`, `revokeRole`, and the overridden `renounceRole` remain externally callable. `grantRole` and `revokeRole` follow OpenZeppelin's role-admin rules; `renounceRole` is modified only to block direct renunciation of `DEFAULT_ADMIN_ROLE`.
- **Upgrades:** Inherited `upgradeToAndCall` is the externally callable UUPS upgrade entry point and is authorized through `_authorizeUpgrade`, which requires `ADMIN_ROLE`. `proxiableUUID` is exposed by the implementation but reverts when called through the proxy.
- **Read-only helpers:** `getRelationship`, `getUserBalances`, `getBatchBalances`, `getActionExecution`, `getRoleMember`, and `getRoleMemberCount` expose application and role-state views.
- **Inherited and generated views:** `hasRole`, `getRoleAdmin`, `supportsInterface`, `paused`, `eip712Domain`, `UPGRADE_INTERFACE_VERSION`, role constants, Permit2 constants, public mappings, and public configuration variables are also externally readable.

4.3 Security-Relevant Design Points

- Upgradeability uses UUPS with `_authorizeUpgrade` restricted to `ADMIN_ROLE`.
- Purchase execution is gated by `RELAYER_ROLE` and user EIP-712 signatures.
- A relayer cannot change the signed payment recipient, token, amount, action ID, content ID, distributor parameters, or maximum gas reimbursement. The relayer does choose whether and when to submit a valid signed action before its deadline, and chooses the actual gas reimbursement charged up to the signed maximum.
- Signed distributor parameters authorize an attempt to create or refresh a fan-creator relationship. The actual distributor used for settlement is the active on-chain relationship at execution time, if one exists.
- The contract does not require distributors to hold a role, appear on an allowlist, be different from the buyer, or be approved by the creator.
- User payment signatures do not bind the current platform fee or platform wallet. Settlement uses the values stored in the vault at execution time.
- `ADMIN_ROLE` is the highest on-chain trust boundary because it can authorize UUPS upgrades to a new implementation.
- Deposits are pulled through the canonical Permit2 address.
- Token accounting assumes supported tokens behave as standard ERC-20 tokens: fixed-balance, non-rebasing, no transfer tax, and no balance-changing side effects outside explicit transfers.
- Emergency withdrawals are intentionally not blocked by pause, but require the admin-controlled `withdrawalsEnabled` flag.
- Accounting relies on `userDeposits` and `totalDeposits`; payment transfers are made directly from vault-held balances.
- Relationship registration follows first-registration-wins semantics while an existing relationship remains active.
- Relayer replacement revokes all current relayers before granting the new relayer.
- Deprecated withdrawal-related storage slots are retained for UUPS storage-layout compatibility.

4.4 Transaction Ordering and MEV Review

The contract has no swaps, oracle-priced trades, auctions, liquidations, or similar mechanics that would normally create sandwich or back-running profit opportunities. Deposits credit the caller, and purchase execution is restricted to `RELAYER_ROLE` plus a user EIP-712 signature, so an arbitrary public mempool searcher cannot directly settle someone else's purchase or redirect vault funds.

The relevant ordering risks are therefore narrower:

- A user can invalidate a pending signed purchase before relayer execution by spending or withdrawing the same recorded balance first. This creates relayer gas-griefing risk and is documented in D-03.
- Admin-controlled state, such as platform fee, platform wallet, pause status, supported-token status, withdrawal enablement, relayer set, or the implementation itself, can change before a signed purchase is settled. The platform-parameter case is covered in F-02, and the broader admin trust boundary is covered in F-03 and F-04.
- Fan-creator relationship registration is order-dependent. If multiple valid signed purchases for the same fan and creator contain different distributor parameters, the first successfully executed registration can determine the active distributor until expiry. Because only the relayer can execute purchases, this is primarily a relayer/backend ordering assumption rather than an external MEV vector.

5. Third-Party Dependency Review

Third-party libraries are not included in the audit scope. This section identifies the external Solidity dependencies used by the in-scope contracts, summarizes public advisories reviewed during the audit, and states whether those advisories appear relevant to the project's imported code paths.

5.1 Third-Party Libraries Used

- OpenZeppelin Contracts 5.4.0, resolved in the lockfile. VimoVault imports ERC-20 interfaces/utilities and ECDSA signature recovery.
- OpenZeppelin Contracts Upgradeable 5.4.0, resolved in the lockfile. VimoVault imports the UUPS, access-control, pausable, reentrancy-guard, and EIP-712 upgradeable modules.
- Uniswap Permit2, through a vendored minimal IAllowanceTransfer interface. The external canonical Permit2 deployment is used at runtime, but the full Permit2 implementation is not included in this audit scope.

The lockfile also contains deployment, testing, and build-tool packages. Those packages are not linked into the deployed Solidity bytecode and are excluded from the smart contract scope.

5.2 Public Advisories Reviewed

OpenZeppelin Bytes utility

[GHSA-9rcw-c2f9-2j55](#) / CVE-2025-54070 describes an out-of-bounds memory access issue in the Bytes `lastIndexOf` utility. The affected range is OpenZeppelin Contracts and Contracts Upgradeable 5.2.0 through versions before 5.4.0; the issue is patched in 5.4.0. This is not applicable to the in-scope code because the project resolves both OpenZeppelin packages to 5.4.0, and VimoVault does not import or call the affected utility.

OpenZeppelin Base64 utility

[GHSA-9vx6-7xxf-x967](#) / CVE-2024-27094 describes an out-of-bounds read in `Base64.encode`. The affected ranges are OpenZeppelin 4.5.0 through versions before 4.9.6, and 5.0.0-rc.0 through versions before 5.0.2; the issue is patched in 4.9.6 and 5.0.2. This is not applicable to the in-scope code because the project resolves to 5.4.0, and VimoVault does not import or call `Base64`.

Snyk package review

The Snyk package page for [@openzeppelin/contracts](#) was reviewed, together with the corresponding OpenZeppelin GitHub security advisories. No direct vulnerability was identified for the resolved 5.4.0 OpenZeppelin packages used by this project.

Uniswap Permit2

The local in-scope code contains only a minimal AllowanceTransfer interface vendored from [Uniswap Permit2](#). The deployed Permit2 implementation and its broader behavior are external third-party infrastructure and are not audited here.

As an additional package-health check, `npm audit --omit=dev` was run against the contract project's lockfile and reported zero production dependency vulnerabilities. This automated result does not replace manual review of the in-scope Solidity code and does not audit third-party library internals.

6. Storage Layout Review

VimoVault is UUPS-upgradeable, so storage layout stability is part of the security review. The source contains explicit storage-layout notes for the contract-owned sequential slots. These notes were checked against the compiler output using Solidity 0.8.24, via IR, and optimizer settings matching the project's Hardhat configuration.

The implementation constructor disables direct initialization, the proxy setup uses an `initializer`, and UUPS upgrade authorization is restricted by `ADMIN_ROLE`. OpenZeppelin's UUPS implementation also checks that a new implementation exposes the expected ERC-1822 `proxiableUUID`. No issue was identified in the current upgrade-authorization path beyond the role-management issue described in F-01.

The storage layout was extracted from the Solidity compiler artifact generated with the project's configured compiler settings.

Slot	Variable	Compiler Layout Result
0	<code>supportedTokens</code>	Matches the comment: supported-token mapping.
1	<code>userDeposits</code>	Matches the comment: user/token balance mapping.
2	<code>totalDeposits</code>	Matches the comment: aggregate token-deposit mapping.
3	<code>withdrawalRequestCount</code>	Matches the comment: deprecated legacy withdrawal state.
4	<code>withdrawalRequests</code>	Matches the comment: deprecated legacy withdrawal state.
5	<code>userWithdrawalRequests</code>	Matches the comment: deprecated legacy withdrawal state.
6	<code>minWithdrawalAmount</code>	Matches the comment: deprecated legacy withdrawal state.
7	<code>emergencyWithdrawalDelay</code>	Matches the comment: deprecated legacy withdrawal state.
8	<code>emergencyWithdrawalTimestamps</code>	Matches the comment: deprecated legacy withdrawal state.
9	<code>_roleMembers</code>	Matches the comment: local role-member enumeration mapping.
10	<code>platformWallet</code>	Matches the comment: platform wallet address.
11	<code>platformFeeBps</code>	Matches the comment: platform fee basis points.
12	<code>executedActions</code>	Matches the comment: action replay-protection mapping.
13	<code>relationships</code>	Matches the comment: fan/creator relationship mapping.
14	<code>actionExecutedAtBlock</code>	Matches the comment: executed-action block mapping.
15	<code>withdrawalsEnabled</code>	Matches the comment: emergency-withdrawal flag.
16	<code>emergencyWithdrawalAmounts</code>	Matches the comment: deprecated legacy withdrawal state.
17–55	<code>__gap</code>	Matches the comment: <code>uint256[39]</code> reserved gap.

The comment's total is also correct: 17 custom slots plus a 39-slot reserved gap equals 56 sequential slots. OpenZeppelin v5 upgradeable modules used by VimoVault declare ERC-7201 namespaced storage locations, so inherited AccessControl, Pausable, ReentrancyGuard, EIP-712, UUPS, and initialization state does not occupy the contract-owned sequential slot range above.

Conclusion: No mismatch was identified between the storage-layout comments and the compiler-reported storage layout for the audited implementation. Future upgrades should keep the deprecated storage slots in place and add new state only immediately before `__gap`, with the gap reduced by the number of newly consumed slots.

7. Compiler and Chain Compatibility

The project Hardhat configuration compiles the audited contract with Solidity 0.8.24, optimizer enabled with 200 runs, and `viaIR = true`. The configuration does not set an explicit `evmVersion`.

The generated artifact metadata reports `evmVersion = cancun`. The deployed bytecode therefore depends on post-Shanghai/Cancun EVM support; for example, the generated runtime bytecode uses `PUSH0`. This is not a problem by itself, but it means chain compatibility must be checked against the target deployment networks.

The repository identifies Polygon, BSC, Polygon Amoy, and BSC testnet as the intended deployment networks. For those networks, no opcode compatibility issue was identified during this review:

- Polygon PoS activated the Napoli hardfork on Polygon mainnet and Amoy. Polygon PIP-33 includes PIP-31, which enables Polygon's Cancun EIP set, including transient storage opcodes, `MCOPY`, and the Cancun `SELFDESTRUCT` semantics.
- BSC's Tycho hardfork documentation states that BSC enables several Ethereum Cancun EIPs, including transient storage opcodes, `MCOPY`, `SELFDESTRUCT` changes, and `BLOBBASEFEE`. The BNB Chain BEP index also marks `PUSH0`, `MCOPY`, and transient storage opcode BEPs as enabled.

Conclusion: The current compiler output is compatible with the repository's stated Polygon and BSC deployment targets, assuming deployment is performed after the relevant network hardfork activations. If VimoVault is later deployed to any additional EVM-compatible chain, that chain should be checked for Shanghai/Cancun opcode support before deployment, or the project should pin a more conservative `evmVersion` such as `paris`.

8. Dynamic Analysis and Tool-Assisted Verification

Targeted execution tests, Foundry fuzz tests, stateful invariants, and external tool sweeps were added to complement the manual review. The Foundry suite used Solidity 0.8.24 with optimizer runs of 200 and via-IR compilation. Fuzz runs were fixed at 1,024; invariant runs were fixed at 256 with depth 32.

The following commands were executed during this pass:

```
1 forge test -vv
2 forge coverage --summary --ir-minimum
3 echidna . --contract EchidnaVaultTest --test-mode assertion --test-limit 5000 --seq-len
  50 --format text
4 echidna . --contract EchidnaVaultTest --test-mode property --test-limit 5000 --seq-len
  50 --format text
5 medusa fuzz --compilation-target . --target-contracts MedusaVaultTest --test-limit 5000
  --seq-len 50 --workers 4 --senders 0xe05fc23807536bEe418f142D19fa0d21BB0cfF7,0
  x10000,0x20000,0x30000 --deployer 0x30000 --no-color --fail-fast
6 slither . --filter-paths 'lib|out|cache|src/test|src/mocks|test' --exclude-dependencies
  --exclude solc-version,pragma,low-level-calls,assembly,naming-convention,immutable-
  states,constable-states,dead-code,unused-state,shadowing-local --checklist
7 aderyn . --src src --path-excludes 'src/test,src/mocks,lib,out,cache'
```

Check	Result
Foundry test suites	4 suites passed; 22 tests passed, 0 failed, 0 skipped
Forge fuzz cases	4 cases, 1,024 runs each
Foundry stateful invariants	2 invariants passed; 256 runs each, 8,192 calls each, 0 reverts
Forge coverage, VimoVault.sol	79.23% lines, 78.41% statements, 14.86% branches, 72.41% functions
Echidna assertion mode	5,167 total calls; all assertion tests passing
Echidna property mode	6 properties passing; 5,139 total calls
Medusa fuzzing	115,421 calls; 24 tests passed, 0 failed; coverage report generated
Slither filtered production scan	2 findings on the <code>_pullAndCredit</code> external-call/accounting pattern, reviewed below
Aderyn filtered production scan	1 high-severity heuristic and 11 low-severity heuristics, reviewed below

The Foundry fuzz and invariant coverage exercised:

- Deposit and emergency-withdrawal accounting over randomized deposit amounts.

- Purchase accounting without a distributor over randomized purchase amounts and gas reimbursements.
- Purchase accounting with an active distributor over randomized purchase amounts, distributor fee basis points, and gas reimbursements.
- Rejection of gas reimbursements above the signed cap, with randomized purchase amounts and cap values.
- Multi-step state sequences combining deposits, emergency withdrawals, purchases with and without distributors, and direct token transfers to the vault.
- Post-sequence invariants that `totalDeposits` equals the modeled sum of user deposits, and that the vault token balance equals tracked deposits plus modeled untracked surplus.

The deterministic execution tests covered:

- Proxy initialization, role assignment, and role membership views.
- Permit2-backed deposits, aggregate deposit accounting, and immediate emergency withdrawals.
- Successful purchase settlement, including platform fee, creator amount, distributor amount, gas reimbursement, vault balance, and aggregate balance updates.
- EIP-712 signature verification, including replay protection, wrong submitter rejection, and rejection when signed `contentId` or `amountDisplay` fields are tampered.
- Pause behavior: deposits and purchases revert while paused, while emergency withdrawal remains available after admin enablement.
- The Slither-reported deposit accounting edge case where a non-standard supported token increases the vault balance during the Permit2 transfer and the whole balance increase is credited to the depositor.
- The acknowledged self-distribution design described in D-01.
- The platform fee and platform wallet timing issue described in F-02.
- The withdrawal-enabled dependency described in F-04.
- The relayer gas-griefing paths documented in D-03, including both user withdrawal before settlement and multiple signed purchases overcommitting one balance.
- First-registration-wins relationship ordering when multiple signed purchases contain different distributor parameters.
- The default-admin self-revocation behavior described in F-01.
- Direct token transfers to the vault creating untracked surplus, supporting the token-assumption discussion in F-07.

The Echidna harness focused on deposit, withdrawal, platform-configuration, and basic blocked-execution properties. The Medusa harness exercised signature-heavy payment paths, including valid purchases, replay rejection, expired signatures, wrong-signer rejection, self-purchase blocking, distribution conservation, and relationship immutability.

Tool-sweep triage:

- Slither reported the external Permit2 call before accounting writes in `_pullAndCredit`. The classic reentrancy concern is mitigated by `nonReentrant`, but the balance-delta accounting behavior remains a Low-severity attribution risk documented in F-05.
- Aderyn reported an Ether-lock heuristic due to inherited upgradeability code. The reviewed OpenZeppelin upgrade path rejects non-payable empty upgrades and only accepts value when an authorized upgrade call executes `setup calldata`; this is an admin-controlled upgrade-safety consideration rather than a direct loss path for token deposits. The remaining Aderyn findings were low-severity style, centralization, loop-cost, opcode, unchecked internal role-return, pragma, and retained-storage-gap heuristics triaged as non-actionable or already reflected in the trust-boundary discussion.

No Critical, High, or Medium issue was identified during this dynamic-analysis and tool-assisted pass. The execution tests and scanner output did confirm several Low and Informational observations included in the findings section, plus the acknowledged design and operational behaviors documented in Section 11.

This testing does not expand the audit scope to third-party token implementations, the external Permit2 implementation, live deployment state, or off-chain frontend/backend/relayer code. Local test doubles were used only to exercise the in-scope contract logic deterministically.

Remediation review verification The remediation review at `ac21c55f` included the following additional checks:

- `npm test` in `contracts/`: 101 Hardhat tests passed. Hardhat warned that the local Node.js v23.10.0 runtime is outside Hardhat's supported Node range; the test suite still completed successfully.
- Targeted Vitest run for payment signing and popup signing flows: 5 test files passed, 102 tests passed. React `act(...)` warnings were emitted by existing popup tests but did not cause failures.
- Slither rescan of the repaired contract still reported the `_pullAndCredit` balance-before-external-call pattern, which is why F-05 remains a Low-severity acknowledged finding rather than being marked resolved. Slither also reported an arbitrary-from issue in the local `MockPermit2` test contract, which is outside the production contract surface.

9. Severity Definitions

Critical	Vulnerabilities that can directly cause loss of funds, permanent contract compromise, or unrecoverable protocol failure.
High	Vulnerabilities that can cause material fund loss, privilege escalation, or major protocol disruption under realistic conditions.
Medium	Issues that can cause limited fund loss, degraded guarantees, blocked workflows, or require specific preconditions to exploit.
Low	Issues with limited direct impact, defensive hardening opportunities, or edge cases that reduce robustness.
Informational	Observations, trust assumptions, documentation gaps, or recommended improvements without immediate exploitable impact.

10. Findings Summary

The following table lists security findings and informational observations that are included in the severity count.

ID	Severity	Status	Title
F-01	Low	Resolved	Default admin can still remove itself through <code>revokeRole</code>
F-02	Low	Resolved	Platform fee and wallet are not bound in user authorizations
F-03	Low	Acknowledged	Admin-controlled UUPS upgrades are the highest trust boundary
F-04	Low	Acknowledged	Emergency withdrawals depend on admin-controlled enablement
F-05	Low	Acknowledged	Deposit balance-delta accounting can credit tokens the depositor did not send
F-06	Informational	Resolved	OpenZeppelin version floor should be raised
F-07	Informational	Acknowledged	Supported token accounting assumptions

The following items are documented separately as acknowledged design or operational decisions. They are not assigned severity and are not included in the severity count.

ID	Type	Title
D-01	Design Decision	Buyer can receive distributor share from its own purchase
D-02	Acknowledged Operational Risk	Relayer-controlled gas reimbursement within signed maximum
D-03	Acknowledged Operational Risk	Users can grief the relayer by invalidating signed purchases before execution

11. Acknowledged Design and Operational Decisions

D-01: Buyer can receive distributor share from its own purchase

Status: Acknowledged as intended design by the project team

Description VimoVault allows any signed, non-zero distributor address to be registered for a fan-creator relationship, provided the distributor fee is within the allowed range and the expiry timestamp is valid. The contract blocks `user == creator`, but it does not block `distributor == user`. There is also no on-chain distributor role, allowlist, creator approval check, or other distributor eligibility check.

As a result, a buyer can be the distributor for its own purchase if the signed payment authorization contains the buyer's address as the distributor. Once that payment is executed, the relationship follows the normal first-registration-wins behavior and can remain active until the signed expiry, up to the contract's 730-day maximum.

Design Impact Self-distribution lets a buyer receive the distributor share back from its own purchase, reducing the buyer's effective cost while preserving the gross payment amount used by the payment flow.

For example, with a 10 USDT purchase, a 10% platform fee, a 40% distributor fee, and no gas reimbursement, settlement becomes:

Recipient	Amount
Platform wallet	1 USDT
Buyer as distributor	4 USDT
Creator	5 USDT

The buyer's net cost is therefore 6 USDT for a 10 USDT purchase, and the creator receives 5 USDT instead of the 9 USDT they would receive with no distributor. Because the relationship is sticky while active, the same buyer can continue to receive the distributor share on later purchases from the same creator until the relationship expires.

Audit Treatment The project team confirmed that this behavior is an intended part of the distribution design. No severity is assigned, and this item is not counted as a security finding. It is retained in the report to make the economic effect and trust assumption explicit for future maintainers, integrators, and reviewers.

D-02: Relayer-controlled gas reimbursement within signed maximum

Status: Acknowledged by the project team; partially hardened in `ac21c55f`

Description `executePurchase` accepts both a reimbursement amount and a maximum reimbursement amount. The user's EIP-712 signature binds the maximum, but it does not bind the exact reimbursement charged in the settlement transaction.

In plain terms, the user signs a ceiling, not the final gas charge. Any account with `RELAYER_ROLE` can choose any reimbursement value up to that signed ceiling when submitting an otherwise valid signed purchase. The contract checks that the charged reimbursement is within the signed maximum and does not exceed the creator's post-fee share, but it does not independently prove that the chosen value equals the relayer's actual gas cost.

Project Response The project team acknowledged this behavior and stated that gas reimbursement selection is handled by off-chain relayer controls. Those off-chain controls were outside the smart contract audit scope and were not independently verified as an on-chain guarantee.

Remediation Review The remediation commit added a protocol-level gas reimbursement cap of `MAX_GAS_REIMBURSEMENT_BPS = 1000`, limiting reimbursement to 10% of the payment amount even if the signed maximum is higher. This is useful defense in depth and reduces the worst-case over-charge bound. It does not prove the exact gas cost on-chain, so the item remains documented as an acknowledged operational risk rather than being converted into a resolved finding.

Residual Risk If the off-chain controls are misconfigured, bypassed, or operated by a compromised relayer, creator proceeds can be reduced up to the user-signed gas reimbursement maximum. This does not let the relayer change the signed purchase recipient, token, amount, action ID, content ID, distributor parameters, or maximum reimbursement, and it does not transfer tokens to an arbitrary relayer address. The remaining risk is bounded settlement unfairness within the signed ceiling.

Audit Treatment Because the project team acknowledged the behavior and relies on off-chain controls for reimbursement fairness, this item is documented as an acknowledged operational risk rather than counted as a security finding.

D-03: Users can grief the relayer by invalidating signed purchases before execution

Status: Acknowledged by the project team; handled through off-chain controls

Description `executePurchase` is submitted by an account with `RELAYER_ROLE`, but it spends the user's recorded vault balance. The user's EIP-712 signature authorizes a payment; it does not reserve that balance inside the contract.

At execution time, the contract checks `userDeposits[user][token] >= amount`. If the user has withdrawn or otherwise spent the same balance before the relayer transaction is mined, the transaction reverts with `Insufficient balance`. Because the relayer submitted the transaction, the relayer pays the native gas cost for the failed execution, and no token-denominated gas reimbursement is paid.

The same pattern can occur if a user signs multiple purchases that are each valid on their own but collectively exceed the user's current balance. The first settlement can succeed, while later settlements revert if the relayer submits them after the balance has been depleted.

Project Response The project team acknowledged this behavior and stated that the relayer side has off-chain handling for these cases. Expected controls include pre-broadcast simulation, balance re-checks, queue management, and retry or backoff rules. Those controls were outside the smart contract audit scope and were not independently verified as on-chain guarantees.

Remediation Review No on-chain reservation or cancellation mechanism was added in `ac21c55f`. The audit treatment therefore remains unchanged: this is an acknowledged operational risk handled by relayer-side controls.

Residual Risk This path does not let the user steal vault funds or receive the gas reimbursement. The risk is operational: failed relayer transactions, native gas loss, queue noise, and possible settlement delays if the off-chain relayer does not strictly filter stale or overcommitted signed actions.

Audit Treatment Because the project team acknowledged the behavior and relies on off-chain relayer controls, this item is documented as an acknowledged operational risk rather than counted as a security finding.

12. Detailed Findings

F-01 (Low): Default admin can still remove itself through `revokeRole`

Status: Resolved in `ac21c55f`

Description The contract is trying to protect itself from a simple mistake: the last default admin should not be able to give up that role and leave the system without a default admin.

The contract currently blocks one way to do that. If the default admin calls `renounceRole` for the default admin role, the call reverts.

The problem is that OpenZeppelin has another role-removal function: `revokeRole`. A default admin is allowed to manage the default admin role itself. So if there is only one default admin, that same account can call `revokeRole` on `DEFAULT_ADMIN_ROLE` and remove its own default admin role anyway.

In plain terms, the front door is locked, but the same action is still possible through another public role-management entry point.

This behavior was reproduced during audit testing: direct renunciation of the default admin role reverts, but self-revocation through `revokeRole` succeeds.

Impact This does not immediately disable all privileged vault operations, because `ADMIN_ROLE` controls the contract-specific admin functions and UUPS upgrade authorization.

The issue is that role management can become permanently impaired. For example, the system may no longer be able to add a new pauser, recover the default admin role, or cleanly fix a future role-configuration mistake. This is why the issue is classified as Low rather than High.

Recommendation If the intended rule is "there must always be at least one default admin", apply that rule to both removal paths. In practice, this means adding a guard to `revokeRole` so the final default admin cannot be removed. Another option is to use an admin model similar to OpenZeppelin's default-admin rules, with an explicit admin-transfer process and delay.

Remediation Resolved. The remediation commit overrides `revokeRole` and blocks removal of a `DEFAULT_ADMIN_ROLE` holder when that removal would leave zero default admins. The fix also adds tests for rejecting last-admin revocation and allowing revocation when another default admin remains. This remediation was reviewed and accepted.

F-02 (Low): Platform fee and wallet are not bound in user authorizations

Status: Resolved in `ac21c55f`

Description The user-signed payment authorization binds the buyer-facing payment parameters: action ID, content ID, creator, token, amount, maximum gas reimbursement, display amount, distributor parameters, expiry, and deadline. It does not bind the current `platformFeeBps` or `platformWallet`.

During settlement, `_distributePayment` reads the current `platformFeeBps` from storage and sends both the platform fee and any gas reimbursement to the current `platformWallet`. Both values can be changed by `ADMIN_ROLE` through `setPlatformFee` and `setPlatformWallet`.

Impact If the platform fee or platform wallet is changed after a user signs a purchase but before the relayer executes it, the final settlement can differ from what was shown to the user or expected by the creator at signing time.

For example, an admin can raise the platform fee up to the contract maximum of 50% before a pending signed payment is executed. The buyer still spends the signed amount, but the creator receives less and the platform receives more. An admin can also change `platformWallet` before execution, causing platform fees and gas reimbursements from already signed purchases to be routed to the new wallet.

This is classified as Low because the behavior requires an admin action, does not let an arbitrary external caller drain vault funds, and the platform fee remains bounded by the contract's fee cap. It is still a settlement-integrity issue: users and creators do not receive an on-chain guarantee that platform parameters at signing time match platform parameters at execution time.

Recommendation If the signed purchase is intended to commit to the exact economic terms shown to the user, include the platform fee and platform wallet, or a platform-parameter version, in the signed payload and verify it during execution. Alternatively, enforce an operational rule that platform fee and platform wallet changes do not take effect until pending signatures have expired.

Remediation Resolved. The remediation commit extends the EIP-712 `PaymentAuthorization` struct with `platformFeeBps` and `platformWallet`. `executePurchase` snapshots the current platform terms, verifies the signature against those values, and settles with the same snapshot. If either platform parameter changes after signing and before execution, the signature no longer verifies and the user must re-sign under the new terms.

The frontend signing flow, on-chain harness, Medusa harness, and contract tests were updated to include the new fields. The fix was validated by Hardhat tests covering platform-fee and platform-wallet mutation between signing and execution.

F-03 (Low): Admin-controlled UUPS upgrades are the highest trust boundary

Status: Acknowledged; no code change planned

Description `VimoVault` is a UUPS-upgradeable vault. The externally callable UUPS upgrade path is authorized by `_authorizeUpgrade`, which requires `ADMIN_ROLE`.

The current implementation does not expose a direct token sweep function, an owner withdrawal function, or a low-level arbitrary-call function. However, an account that can authorize upgrades can replace the implementation with new logic that changes withdrawal rules, changes accounting, or transfers vault-held tokens outside the current purchase and emergency-withdrawal paths.

Impact The admin-controlled upgrade path is the strongest on-chain trust boundary in the system. A compromised or malicious upgrade admin can change the implementation and gain practical control over vault-held assets and future settlement logic.

This is classified as Low in the context of the current code review because it is an expected property of UUPS upgradeability rather than an implementation bug in the current contract. It must nevertheless be treated as a material operational security assumption for any deployed vault that holds user balances.

Recommendation Protect upgrade authority with production-grade governance controls, such as a multisig, separation between operational roles and upgrade authority, public upgrade monitoring, and a time delay or review window before upgrades take effect. Users and integrators should understand that the upgrade admin is trusted not to deploy malicious or unsafe implementations.

Remediation Acknowledged / will not fix. No code remediation was reviewed in ac21c55f. The project accepts this as an operational trust-boundary property of the UUPS upgrade model.

F-04 (Low): Emergency withdrawals depend on admin-controlled enablement

Status: Acknowledged; no code change planned

Description `executeEmergencyWithdrawal` is the user's direct exit path for recorded vault balances. The function is intentionally not gated by `whenNotPaused`, so users can still exit while the main deposit and purchase flows are paused.

However, emergency withdrawal is still gated by `withdrawalsEnabled`. The initializer leaves this flag disabled by default, and `ADMIN_ROLE` can enable or disable it through `setWithdrawalsEnabled`.

Impact If `withdrawalsEnabled` is false, users cannot directly withdraw their recorded balances through the emergency path. This can matter during the exact conditions where an emergency exit is most important, such as a paused contract, relay outage, backend incident, or disabled purchase flow.

This does not let an arbitrary account steal funds. It is a liveness and user-exit risk: users depend on an admin action or admin policy to keep the exit path available. The risk is amplified if withdrawals are disabled at the same time another operational issue prevents normal settlement or user activity.

Recommendation If emergency withdrawal is intended to be a reliable user failsafe, keep it enabled after launch and treat disabling it as an exceptional incident response action. Consider adding operational safeguards such as public monitoring of the flag, documented runbooks, multisig approval for disabling withdrawals, or a delay/notice period before withdrawals can be disabled in non-emergency conditions. If the enablement gate is only needed for deployment rollout, consider removing or permanently enabling it once the system is live.

Remediation Acknowledged / will not fix. No code remediation was reviewed in ac21c55f. The project accepts that the emergency withdrawal path remains dependent on the admin-controlled `withdrawalsEnabled` flag.

F-05 (Low): Deposit balance-delta accounting can credit tokens the depositor did not send

Status: Acknowledged as intended design by the project team

Description `deposit` and `permitAndDeposit` both call `_pullAndCredit`. That helper records the vault's token balance before the `Permit2` transfer, asks `Permit2` to pull tokens from the user, then credits the user with the full difference between the vault's token balance before and after that external call.

In plain terms, the contract does not credit "the exact amount taken from this user". It credits "whatever amount the vault balance increased by while this deposit was running".

For standard USDT/USDC-style tokens, those two values should be the same. The risk appears when a non-standard, malicious, or otherwise unexpected supported token can increase the vault's balance during its `transferFrom` path. In that case, the vault has no way to tell which tokens came from the depositor and which tokens came from somewhere else during the same external call. The whole increase is credited to the depositor.

Slither flagged this as an external-call/accounting pattern in `_pullAndCredit`. A focused Foundry test reproduced the accounting edge case with a non-standard token: the user deposited 100 units, the token caused an extra 50 units to arrive at the vault during the `Permit2` transfer, and the vault credited the user with 150 units.

Impact This is not the usual "reenter the vault and drain everything" issue. Both deposit paths use the reentrancy guard, so a token callback cannot simply enter the same vault flow again.

The issue is balance attribution. If an unsupported or malicious token is never allowlisted, there is no practical impact. If such a token is mistakenly allowlisted, a depositor may receive vault credit for tokens they did not personally send. That credit can then be used like any other recorded balance, including for purchases or for emergency withdrawal when withdrawals are enabled.

Worst Case The worst realistic case is a bad token allowlist decision. An admin adds a non-standard or malicious token as supported. An attacker deposits a small amount of that token, while the token's transfer behavior causes additional tokens to be sent to the vault during the same transfer. The vault credits the attacker for the full balance increase, not just the attacker's own transfer. The attacker can then withdraw or spend that inflated credit, effectively claiming token balance that should have remained unassigned or belonged to another source.

For the intended standard USDT/USDC deployment set, this is a Low-severity hardening finding rather than a current direct drain. It becomes more serious if the supported-token allowlist is expanded without strict token-behavior review.

Recommendation Keep the supported-token list restricted to vetted, standard, non-rebasing, no-fee ERC-20 stablecoins. Treat any token with hooks, transfer fees, reflection, rebasing, automatic balance changes, or custom transfer side effects as incompatible unless the deposit accounting model is redesigned.

For stronger defense in depth, consider crediting exactly the requested amount for known standard tokens and separately tracking unexpected surplus, or adding an explicit post-transfer check that `received == amount`. If fee-on-transfer tokens ever need to be supported, that should be a separate code path with token-specific accounting and clear surplus handling.

Project Response The project team acknowledged this behavior as by design. The design relies on the supported-token allowlist being restricted to vetted, standard, non-rebasing, no-fee ERC-20 stablecoins without balance-changing side effects.

Remediation Acknowledged by design. No code change altering the `_pullAndCredit` balance-delta accounting model was reviewed in `ac21c55f`. The Slither remediation rescan still reports the same external-call/accounting pattern, so this item remains a Low-severity acknowledged finding.

F-06 (Informational): OpenZeppelin version floor should be raised

Status: Resolved in `ac21c55f`

Description The contract lockfile currently resolves both OpenZeppelin packages to version 5.4.0, which is not affected by the public OpenZeppelin advisories reviewed in Section 5. However, the package manifest still declares both OpenZeppelin dependencies as `^5.0.1`. That version constraint does not communicate the intended patched minimum version in the project manifest.

This matters because OpenZeppelin 5.0.1 itself is within the affected range for the Base64 advisory discussed in Section 5, and OpenZeppelin 5.2.x / 5.3.x are within the affected range for the Bytes utility advisory discussed in Section 5. The current lockfile is therefore acceptable, but the dependency policy would be clearer and more resilient if the manifest itself required OpenZeppelin 5.4.0 or higher.

Impact No direct vulnerability was identified in the currently resolved dependencies. This is a dependency hygiene issue: future lockfile regeneration, manual dependency changes, or CI workflows that do not strictly install from the lockfile could reintroduce a version range that includes known affected OpenZeppelin releases.

If a future UUPS implementation or contract extension were accidentally built against an affected OpenZeppelin release and then imported one of the affected utilities, the upgraded implementation

could deploy code with a known out-of-bounds memory read/access issue. This would not be caused by the current VimoVault code path, but it is exactly the kind of avoidable dependency drift that the manifest should prevent.

Recommendation Update the OpenZeppelin dependency declarations to require 5.4.0 or higher, for example `^5.4.0`, and continue committing and enforcing the lockfile. Production builds should use deterministic installs, such as `npm ci`, and CI should include a package-audit check for production dependencies.

Remediation Resolved. The remediation commit updates both `contracts/package.json` and `contracts/package-lock.json` so both OpenZeppelin packages require `^5.4.0`. The resolved lockfile remains on 5.4.0.

F-07 (Informational): Supported token accounting assumptions

Status: Acknowledged assumption

Description VimoVault is designed around stablecoin deposits and payment distribution. The codebase indicates deployment on Polygon, BSC, Polygon Amoy, and BSC testnet, and the deployment script configures USDT and USDC as the default supported assets on Polygon and BSC.

Those are reasonable target assets for this accounting model. Public ecosystem data also shows active stablecoin markets on both chains: Polygon's developer documentation describes broad stablecoin support and identifies native USDC as a common payment stablecoin, while DeFiLlama's stablecoin-by-chain data shows BSC and Polygon as large stablecoin venues dominated by USDT and USDC respectively.

The contract's internal accounting assumes that supported tokens behave like standard fixed-balance ERC-20 tokens. Deposits credit the actual amount received, payment execution debits the user's recorded balance, and outgoing transfers are assumed to deliver the requested amount to the recipient.

Impact No direct issue was identified for standard USDT/USDC-style ERC-20 stablecoins. The contract does not depend on `decimals()`, so token decimals are mainly a front-end, back-end, and relay unit-conversion concern. Fee calculations use integer division and round down in the usual Solidity way; any remainder stays in the creator's share. Solidity 0.8 checked arithmetic, together with the current fee bounds and balance checks, makes overflow or underflow unlikely in normal operation.

Non-standard tokens can break these assumptions. A negative rebasing token can make the vault's real token balance lower than the recorded user balances, causing withdrawals or purchases to fail. A positive rebasing token, unsolicited transfer, or other balance-increasing behavior can create surplus that is not assigned to any user. A fee-on-transfer, burn-on-transfer, or reflection token can also make outgoing distributions misleading, because the contract records and emits the requested transfer amount even if the recipient receives less.

Recommendation Keep `supportedTokens` as a strict allowlist of vetted token contract addresses for each deployed chain. Before adding a token, confirm that it is a standard non-rebasing ERC-20 with no transfer fee, burn, reflection, hook, or other balance-changing side effect. Rebasing, fee-on-transfer, synthetic yield-bearing, or other non-standard stablecoins should not be supported unless the vault accounting model is explicitly redesigned for them.

Remediation No code change was required for the intended standard USDT/USDC deployment set. The assumption remains important for future token allowlist expansion.